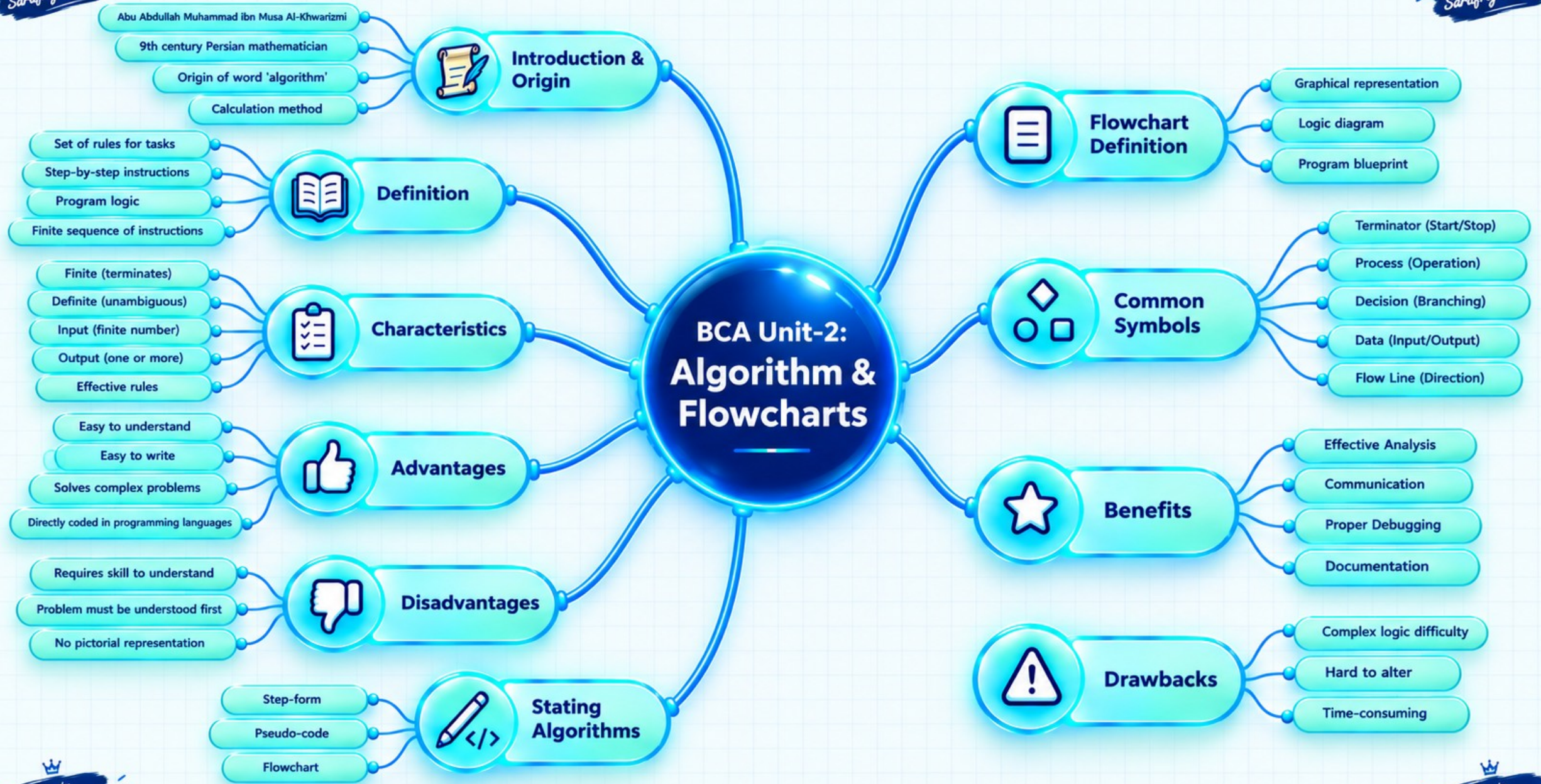




2.1 ALGORITHM



Introduction & Origin

Introduction

The word “algorithm” is the name of Abu Abdullah Muhammad ibn Musa Al-Khwarizmi who was a Persian mathematician in 9th century. An algorithm is a set of rules which can be used to do some task either manually or electronically (computer system) is known as an algorithm. Most commonly, this term is used with reference to computers. Generally algorithms are used to provide solutions of different problems. Algorithms provide effective methods for problem solving. It solves a problem with a finite sequence of instructions.

Origin

The term “algorithm” is derived from the name of the great Persian mathematician Abu Abdullah Muhammad ibn Musa Al-Khwarizmi, who wrote a book on algebra in the 9th century. His name is often considered the origin of the word “algorithm”.

★ Key Points

- Algorithm is the name of Abu Abdullah Muhammad ibn Musa Al-Khwarizmi.
- It is a set of rules to solve a task.
- Commonly used in computers for problem solving.
- Provides effective methods and finite sequence of instructions.



Al-Khwarizmi



2.1.2 Meaning and Definition



Definition

The term “Algorithm” is taken from a Latin word “algorismus”. The meaning of this word is “calculation method”. To solve a problem specific set of instructions are used, this set of instruction is known as algorithm. For a given problem it is a step by step instruction for how to reach at the solution of that problem. Basically algorithm shows the logic of the program. An algorithm is defined as a finite set of well defined instructions for problem solving. It takes a valid input and produces desired output in a finite amount of time. It is illustrated in figure 2.1.

★ Key Points

- An algorithm is generally used for calculation and processing data using a sequence of finite instructions.
- An algorithm provides step-by-step procedure for problem solving.
- According to Stone and Knuth “an algorithm is a set of rules that precisely defines a sequence of operations in such a way that each rule is effective and definite and that the sequence terminates in a finite time.” An algorithm has a specific format which is shown in figure 2.2.



Figure 2.1: Algorithm

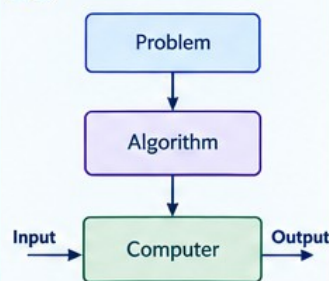


Figure 2.1: Algorithm



An algorithm is generally used for calculation and processing data using a sequence of finite instruction set.



Disadvantages of Algorithms

- Occupy more skill to understand it.
- Problem should be understand before proceed for algorithm.
- Cannot be represented in pictorial form.



Examples

Example 1: Find the area and circumference of a circle of radius r.

Algorithm

- Step 1: Read Radius of Circle: as R
- Step 2: Assign Value of PI = 3.14
- Step 3: Calculate Area = $\text{PI} * R * R$
- Step 4: Calculate Circumference Circum = $2 * \text{PI} * R$
- Step 5: Print Value of Area and Circum
- Step 6: END / STOP



Example 2

Find the area and circumference of a circle of radius r.

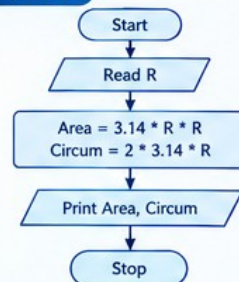


Algorithm

- Step 1: Read Radius of Circle as R
- Step 2: Assign Value of PI = 3.14
- Step 3: Calculate Area = $\text{PI} * R * R$
- Step 4: Calculate Circumference Circum = $2 * \text{PI} * R$
- Step 5: Print Value of Area and Circum
- Step 6: END / STOP



Flowchart



C Program

```
#include <stdio.h>
#include <math.h>
int main() {
    float R, PI = 3.14, Area, Circum;
    printf("Enter radius of circle: ");
    scanf("%f", &R);
    Area = PI * R * R;
    Circum = 2 * PI * R;
    printf("Area = %.2f\n", Area);
    printf("Circumference = %.2f\n", Circum);
    return 0;
}
```



Output

Enter radius of circle: 5
Area = 78.50
Circumference = 31.40



Explanation

The program takes the radius as input, calculates the area and circumference using the given formulas and displays the result.



C Program

```
#include <stdio.h>
#include <math.h>
int main() {
    float R, PI = 3.14, Area, Circum;
    printf("Enter radius of circle: ");
    scanf("%f", &R);
    Area = PI * R * R;
    Circum = 2 * PI * R;
    printf("Area = %.2f\n", Area);
    printf("Circumference = %.2f\n", Circum);
    return 0;
}
```



Output

Enter radius of circle: 5
Area = 78.50
Circumference = 31.40



Explanation

The program takes the radius as input, calculates the area and circumference using the given formulas and displays the result.

2.1 Characteristics of Algorithm

1



Finite

An algorithm must have finite number of steps after which it would be terminated.

2



Definite

Algorithms must not have any ambiguity. Each step of the algorithm must be well-defined and unambiguous.

3



Input

An algorithm can have finite number of inputs.

4



Output

An algorithm can have one or more outputs.

5



Effective

An algorithm must be effective and can be implemented with a finite amount of time.

1



Advantages of Algorithms

- 1 Provides step-by-step procedure to understand easily.
- 2 Algorithm is easy to write.
- 3 Solves those problems which are complex and not solve by simple methods.
- 4 Algorithm in general, can be directly coded in a programming language.

2



Disadvantages of Algorithms

- 1 Occupy more skill to understand it.
- 2 Problem should be understand before proceed for algorithm.
- 3 Cannot be represented in pictorial form.



Example 8: Algorithm of making tea.

Algorithm

- START**
- Step 1:** Assemble tools – cups, saucers, spoons, milk, kettle and teapot.
- Step 2:** Count the number of persons (one or two or more).
- Step 3:** Add 1 spoon of tea leaves to teapot (and 1/2 spoon for every extra person).
- Step 4:** Add water to kettle.
- Step 5:** Turn on heat source.
- Step 6:** Heat kettle water to boiling.
- Step 7:** Add water to teapot.
- Step 8:** Leave tea to brew.
- Step 9:** Pour tea into teacups.
- Step 10:** If milk wanted, add milk.
- Step 11:** If sugar wanted, add sugar.
- Step 12:** Drink and enjoy!
- Step 13:** STOP

C Program

```
void main() {
    int n;
    printf("Enter number of persons: ");
    scanf("%d", &n);

    printf("Enter 1/2 spoon of tea leaves: ");
    scanf("%d", &n); // (simplified input)

    printf("Add water to kettle.\n");
    printf("Turn on heat source.\n");
    printf("Heat kettle water to boiling.\n");
    printf("Add water to teapot.\n");
    printf("Leave tea to brew.\n");
    printf("Pour tea into teacups.\n");
    printf("If milk wanted, add milk.\n");
    printf("If sugar wanted, add sugar.\n");
    printf("Drink and enjoy!\n");
}
```

Explanation

This program reads the number of persons and follows the same steps given in the algorithm to make tea. It only displays the steps on the screen.

Example 9: Algorithm to find the largest of three numbers.

Algorithm

- START**
- Step 1:** Read first number.
- Step 2:** Read second number.
- Step 3:** Read third number.
- Step 4:** Compare and find the largest.
- Step 5:** Display the largest number.
- Step 6:** STOP

C Program

```
void main() {
    int a, b, c, largest;
    printf("Enter first number: ");
    scanf("%d", &a);
    printf("Enter second number: ");
    scanf("%d", &b);
    printf("Enter third number: ");
    scanf("%d", &c);

    largest = a;
    if (b > largest) largest = b;
    if (c > largest) largest = c;
    printf("Largest number is: %d\n",
    return 0;
}
```

Explanation

The program reads three numbers and compares them using conditional statements to find the largest number. The result is then displayed on the screen.

Example 10: Write the steps for calculating the average of two numbers.

Algorithm

- START**
- Step 1:** Read principal, rate and time.
- Step 2:** Calculate $P = P \times R \times T$.
- Step 3:** Calculate $S = Q / 100$.
- Step 4:** Display S.
- Step 5:** STOP

Explanation

The program takes principal, rate and time as input, calculates the simple interest using the given formula and displays the result.

C Program

```
#include <stdio.h>
void main() {
    float prin, num1, num2, avg;
    printf("Enter principal: ");
    scanf("%f", &num1);
    printf("Enter rate: ");
    scanf("%f", &num2);
    avg = (num1 + num2) / 2.0;
    printf("Average = %.2f\n", avg);
    return 0;
}
```

Example 11: Write an algorithm to lock an open room.

Algorithm

- Step 1:** Take the key of the lock.
- Step 2:** Closed the door.
- Step 3:** Put the key in the lock.
- Step 4:** Turn the key clockwise direction.
- Step 5:** Remove the key from the lock.
- Step 6:** STOP

C Program

```
void main() {
    printf("This is a physical action.");
    printf("Locking an open room cannot
    be implemented in C.");
    return 0;
}
```

Explanation

This is a real-world procedure to lock a room. It involves physical actions and cannot be implemented as a C program.



Example 12: Finding the leap Year.

Question Write a program to check whether a given year is a leap year or not.

Algorithm

START

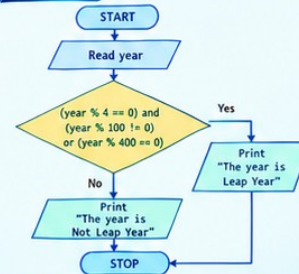
Step 1: Read year.

Step 2: If (year % 4 == 0) and (year % 100 != 0) or (year % 400 == 0) then print "The year is Leap Year";

Step 3: Else print "The year is Not Leap Year".

STOP

Flowchart



C Program

```
#include <stdio.h>
void main() {
    int year;
    printf("Enter year: ");
    scanf("%d", &year);
    if ((year%4==0 && year%100!=0) || (year%400==0))
        printf("The year is Leap Year");
    else
        printf("The year is Not Leap Year");
}
```

Output

Enter year: 2024
The year is Leap Year

Explanation

The program checks the given year for the leap year condition using the given formula and displays the result.

Example 13: Addition of Two Integers.

Question Write a program to add two integers and display the sum.

Algorithm

START

Step 1: Initialize a, b, and z.

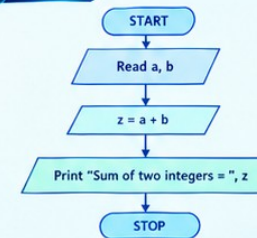
Step 2: Read a, b.

Step 3: z = a + b.

Step 4: Print "Sum of two integers".

Step 5: STOP

Flowchart



C Program

```
#include <stdio.h>
void main() {
    int a, b, z;
    printf("Enter first number: ");
    scanf("%d", &a);
    printf("Enter second number: ");
    scanf("%d", &b);
    z = a + b;
    printf("Sum of two integers = %d", z);
}
```

Output

Enter first number: 52
Enter second number: 30
Sum of two integers = 82

Explanation

The program reads two integers, adds them and displays the sum using the given formula.

Example 14: To choose the largest of two distinct numbers.

Algorithm Write the program to find the largest of two distinct numbers.

Algorithm

START

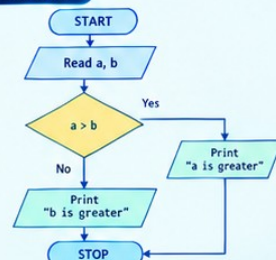
Step 1: Initialize a, b.

Step 2: Read a and b.

Step 3: If (a > b) print "a is greater"; else print "b is greater".

Step 4: STOP

Flowchart



C Program

```
#include <stdio.h>
void main() {
    int a, b;
    printf("Enter first number: ");
    scanf("%d", &a);
    printf("Enter second number: ");
    scanf("%d", &b);
    if (a > b)
        printf("a is greater");
    else
        printf("b is greater");
}
```

Output

Enter first number: 45
Enter second number: 28
a is greater

Explanation

The program compares the two numbers using a conditional statement and displays the larger number.

Example 15: Find the sum of first n natural numbers.

Algorithm Write a program to find the sum of first n natural numbers.

Algorithm

START

Step 1: Initialize SUM = 0 and i = 0.

Step 2: Read N.

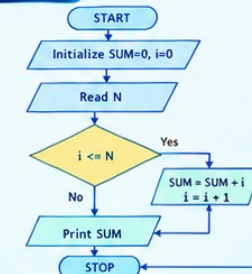
Step 3: Check i <= N. If Yes Then SUM = SUM + i.

Step 4: Increment i = i + 1.

Step 5: Else Print SUM.

Step 6: STOP

Flowchart



C Program

```
#include <stdio.h>
void main() {
    int n, i, sum=0;
    printf("Enter n: ");
    scanf("%d", &n);
    for(i=1; i<=n; i++)
        sum = sum + i;
    printf("Sum = %d", sum);
}
```

Output

Enter n: 50
Sum = 1275

Explanation

The program adds numbers from 1 to n using a loop and displays the sum.